

山东大学



实验五实验报告 Experiment report

组员 1：熊梓刚、组员 2：高焜、组员 3：马旭恩、组员 4：陈奕顺

组员 1 学号：202300460048	班级：网安一班
组员 2 学号：202300460028	班级：密码一班
组员 3 学号：202300460030	班级：网安一班
组员 4 学号：202300460047	班级：密码一班

声明

本实验主要由熊梓刚、高焜、马旭恩、陈奕顺完成。

熊梓刚完成的部分：方案设计，CKKS 参数配置与密钥生成，密文卷积核心算法实现与调试，性能优化。

高焜完成的部分：测试数据准备，明文卷积结果计算，误差分析与精度验证，结果对比。

马旭恩完成的部分：实验环境搭建，编译脚本编写，运行输出记录与截图整理。

陈奕顺完成的部分：实验报告撰写，图表绘制与结果汇总，文档格式校对。

2026 年 7 月 24 日

1 实验基本信息

实验五为基于 OpenFHE 的密文卷积基线方案实现。实验采用 OpenFHE v1.5.1 同态加密库，加密方案为 CKKS。多项式环维度设定为 $N = 32768$ ，SIMD 批处理大小为 16 个槽位。乘法深度设为 3，缩放模数位数为 50 bits。编译环境为 g++ 11.4.0，遵循 C++17 标准，操作系统为 Ubuntu 22.04。

2 实验问题

2.1 问题描述

本实验选用 OpenFHE，使用单输入单输出的 4×4 输入矩阵和 3×3 卷积核，设置步长为 1、填充为 0，在密文域上实现标准二维卷积操作，并验证解密结果与明文卷积结果的一致性。

2.2 测试数据

4×4 输入矩阵：

$$I = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \\ 13 & 14 & 15 & 16 \end{bmatrix} \quad (1)$$

3×3 卷积核：

$$K = \begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix} \quad (2)$$

2.3 期望输出

由于输入为 4×4 ，核为 3×3 ，步长为 1，无填充：

$$\text{输出尺寸} = \left\lfloor \frac{4 - 3 + 0}{1} \right\rfloor + 1 = 2 \times 2 \quad (3)$$

逐元素验证：

$$O[0,0] = 1 \cdot 1 + 2 \cdot 0 + 3 \cdot (-1) + 5 \cdot 1 + 6 \cdot 0 + 7 \cdot (-1) + 9 \cdot 1 + 10 \cdot 0 + 11 \cdot (-1) = -6 \quad (4)$$

$$O[0,1] = 2 \cdot 1 + 3 \cdot 0 + 4 \cdot (-1) + 6 \cdot 1 + 7 \cdot 0 + 8 \cdot (-1) + 10 \cdot 1 + 11 \cdot 0 + 12 \cdot (-1) = -6 \quad (5)$$

$$O[1,0] = 5 \cdot 1 + 6 \cdot 0 + 7 \cdot (-1) + 9 \cdot 1 + 10 \cdot 0 + 11 \cdot (-1) + 13 \cdot 1 + 14 \cdot 0 + 15 \cdot (-1) = -6 \quad (6)$$

$$O[1,1] = 6 \cdot 1 + 7 \cdot 0 + 8 \cdot (-1) + 10 \cdot 1 + 11 \cdot 0 + 12 \cdot (-1) + 14 \cdot 1 + 15 \cdot 0 + 16 \cdot (-1) = -6 \quad (7)$$

因此期望输出为：

$$O = \begin{bmatrix} -6 & -6 \\ -6 & -6 \end{bmatrix} \quad (8)$$

3 解决方案与步骤

本实验采用 OpenFHE CKKS 方案实现密文卷积。CKKS 是一种支持浮点数近似计算的层级全同态加密方案，天然支持 SIMD 单指令多数据打包和密文旋转操作，非常适合于卷积等并行计算场景。

3.1 整体流程

1. **建立 CryptoContext**: 配置 CKKS 参数，包括环维度、批处理大小、乘法深度等
2. **生成密钥**: 生成公私钥对、乘法密钥、旋转密钥
3. **SIMD 打包**: 将 4×4 矩阵按行主序展开为 16 槽位的明文向量
4. **加密**: 将打包后的明文加密为密文
5. **密文卷积**: 执行“旋转 $\rightarrow$ 乘权重 $\rightarrow$ 累加”的核心同态运算
6. **解密**: 解密密文得到结果向量
7. **验证**: 提取 2×2 输出并与期望值对比

3.2 Step 1: 建立 CKKS CryptoContext

CKKS 方案的参数配置决定了加密的安全性、计算能力和精度。

Listing 1: 建立 CKKS CryptoContext

```
1 uint32_t multDepth = 3;
2 uint32_t scaleModSize = 50;
3 uint32_t batchSize = 16;
4
5 CParams<CryptoContextCKKSRNS> parameters;
6 parameters.SetMultiplicativeDepth(multDepth);
7 parameters.SetScalingModSize(scaleModSize);
8 parameters.SetBatchSize(batchSize);
9 parameters.SetRingDim(1 << 15); // 32768
10
11 CryptoContext<DCRTPoly> cc = GenCryptoContext(parameters);
12
13 // Enable necessary features
14 cc->Enable(PKE);
15 cc->Enable(KEYSWITCH);
16 cc->Enable(LEVELEDSE);
17 cc->Enable(ADVANCEDSE);
```

参数解释:

- RingDim = 32768: 多项式环 $R_q = \mathbb{Z}_q[X]/(X^N + 1)$ 的维度 N 。 N 越大安全性越高，但计算开销也越大。32768 足以支持 16 个 SIMD 槽位和所需的旋转操作。
- MultiplicativeDepth = 3: 密文乘法深度上限。每次乘法消耗 1 层深度。本实验最多 1 层乘法，因为乘以权重后再累加不需额外深度，3 层充足。

- ScaleModSize = 50 bits: CKKS 的缩放因子 Δ 的模数位数，影响浮点精度。50 bits 提供约 15 位十进制精度。
- BatchSize = 16: SIMD 打包的槽位数，恰好容纳 4×4 矩阵的所有元素。

3.3 Step 2: 生成密钥

除基础公私钥外，还需生成用于同态乘法的乘法密钥和用于同态旋转的旋转密钥。

Listing 2: 生成密钥

```
1 auto keyPair = cc->KeyGen();
2
3 // 乘法密钥：用于同态乘法后的重线性化
4 cc->EvalMultKeyGen(keyPair.secretKey);
5
6 // 旋转密钥：为每个需要的偏移量预生成密钥
7 std::vector<int32_t> rotationIndices = {1, 2, 4, 5, 6, 8, 9, 10};
8 cc->EvalRotateKeyGen(keyPair.secretKey, rotationIndices);
```

关键点：旋转密钥 $\{1, 2, 4, 5, 6, 8, 9, 10\}$ 必须在使用 EvalRotate 之前预生成。偏移量 0 即原始数据不需要旋转密钥。每个旋转密钥对应一次 Galois 自同构。

3.4 Step 3: SIMD 打包与加密

将 4×4 矩阵按行主序线性化为 16 维向量，然后打包为单个明文并加密。

Listing 3: SIMD 打包与加密

```
1 std::vector<double> input4x4 = {
2     1.0, 2.0, 3.0, 4.0,
3     5.0, 6.0, 7.0, 8.0,
4     9.0, 10.0, 11.0, 12.0,
5     13.0, 14.0, 15.0, 16.0
6 };
7
8 Plaintext plaintextInput = cc->MakeCKKSPackedPlaintext(input4x4);
9 auto ciphertextInput = cc->Encrypt(keyPair.publicKey, plaintextInput);
```

SIMD 打包过程如下：

$$\begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \\ 13 & 14 & 15 & 16 \end{bmatrix} \xrightarrow{\text{行主序展开}} [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16] \xrightarrow{\text{CKKS Pack}} \text{密文ct}$$

3.5 Step 4: 密文卷积核心算法

3.5.1 偏移量与卷积核权重的对应关系

在行主序打包的 4×4 向量中，矩阵位置 (i, j) 对应槽位索引 $i \times 4 + j$ 。 3×3 卷积核的 9 个位置对应的偏移量如下：

表 1: 卷积核位置与偏移量映射

核位置 (r, c)	偏移量 $r \times 4 + c$	权重 $K[r][c]$	说明
(0,0)	0	+1	原位, 无需旋转
(0,1)	1	0	零权重, 可跳过
(0,2)	2	-1	需旋转偏移 2
(1,0)	4	+1	需旋转偏移 4
(1,1)	5	0	零权重, 可跳过
(1,2)	6	-1	需旋转偏移 6
(2,0)	8	+1	需旋转偏移 8
(2,1)	9	0	零权重, 可跳过
(2,2)	10	-1	需旋转偏移 10

3.5.2 算法原理

设密文 ct 的槽位 i 存储输入值 s_i 。执行 $\text{EvalRotate}(ct, \text{offset})$ 后, 槽位 i 获得原槽位 $i + \text{offset}$ 的值。

将旋转后的密文乘以核权重, 所有 9 个乘积累加, 槽位 0 恰好为卷积输出 $O[0, 0]$:

$$\text{result}[0] = \sum_{k=0}^8 \text{Rotate}(ct, \delta_k)[0] \cdot K[r_k][c_k] = \sum_{k=0}^8 s_{\delta_k} \cdot K[r_k][c_k] = O[0, 0] \quad (9)$$

同理:

- $\text{result}[1] = O[0, 1]$
- $\text{result}[4] = O[1, 0]$
- $\text{result}[5] = O[1, 1]$

3.5.3 核心实现代码

Listing 4: 密文卷积核心循环

```

1 std::vector<int32_t> offsets = {0, 1, 2, 4, 5, 6, 8, 9, 10};
2 std::vector<double> kernel3x3 = {1.0, 0.0, -1.0, 1.0, 0.0, -1.0, 1.0, 0.0, -1.0};
3
4 Ciphertext<DCRTPoly> result;
5 bool first = true;
6
7 for (size_t k = 0; k < offsets.size(); k++) {
8     int32_t offset = offsets[k];
9     double weight = kernel3x3[k];
10
11     // Skip zero-weight kernels
12     if (std::abs(weight) < 1e-10) continue;
13
14     // Step A: Rotate ciphertext by offset
15     Ciphertext<DCRTPoly> rotated = (offset == 0)

```

```
16     ? ciphertextInput
17     : cc->EvalRotate(ciphertextInput, offset);
18
19     // Step B: Multiply by kernel weight constant plaintext
20     Plaintext weightPt = cc->MakeCKKSPackedPlaintext(
21         std::vector<double>(batchSize, weight));
22     auto weighted = cc->EvalMult(rotated, weightPt);
23
24     // Step C: Accumulate
25     result = first ? weighted : cc->EvalAdd(result, weighted);
26     first = false;
27 }
```

3.6 Step 5: 解密与结果提取

Listing 5: 解密与结果提取

```
1 Plaintext plaintextResult;
2 cc->Decrypt(keyPair.secretKey, result, &plaintextResult);
3 plaintextResult->SetLength(16);
4 std::vector<double> decrypted = plaintextResult->GetRealPackedValue();
5
6 // Extract 2x2 output: positions 0, 1, 4, 5
7 double 000 = decrypted[0], 001 = decrypted[1];
8 double 010 = decrypted[4], 011 = decrypted[5];
```

4 实验结果

4.1 运行输出

Listing 6: 实验完整运行输出

```
=====
Experiment 5: Baseline HE Convolution (OpenFHE CKKS)
=====

[Step 1] Setting up CKKS CryptoContext...
Ring dimension: 32768
Batch size:    16
Mult depth:    3
Scale mod size: 50 bits

[Step 2] Generating keys...
Rotation keys generated for indices: 1 2 4 5 6 8 9 10

[Step 3] Preparing input data...
Input (4x4):
    1    2    3    4
```

```
5   6   7   8
9  10  11  12
13 14  15  16
```

Kernel (3x3):

```
1   0  -1
1   0  -1
1   0  -1
```

[Step 4] Packing and encrypting input...

Input packed into 16 slots and encrypted.

[Step 5] Performing homomorphic convolution...

```
Kernel[0][0] offset=0, weight=1 -> no rotation needed, multiplied
Kernel[0][1] weight=0 -> skipping zero weight
Kernel[0][2] offset=2, weight=-1 -> rotated, multiplied, added
Kernel[1][0] offset=4, weight=1 -> rotated, multiplied, added
Kernel[1][1] weight=0 -> skipping zero weight
Kernel[1][2] offset=6, weight=-1 -> rotated, multiplied, added
Kernel[2][0] offset=8, weight=1 -> rotated, multiplied, added
Kernel[2][1] weight=0 -> skipping zero weight
Kernel[2][2] offset=10, weight=-1 -> rotated, multiplied, added
```

Operation counts (baseline):

```
Rotations:      5
Multiplications: 6
Additions:      5
```

[Step 6] Decrypting and verifying...

Decrypted result (first 16 slots):

```
-6.0000 -6.0000 -6.0000 -6.0000
-6.0000 -6.0000 10.0000 10.0000
-6.0000 -6.0000 10.0000 10.0000
-6.0000 -6.0000 10.0000 10.0000
```

Extracted 2x2 convolution output:

```
[-6.000000, -6.000000]
[-6.000000, -6.000000]
```

Expected output:

```
[-6.0, -6.0]
[-6.0, -6.0]
```

[Step 7] Verification...

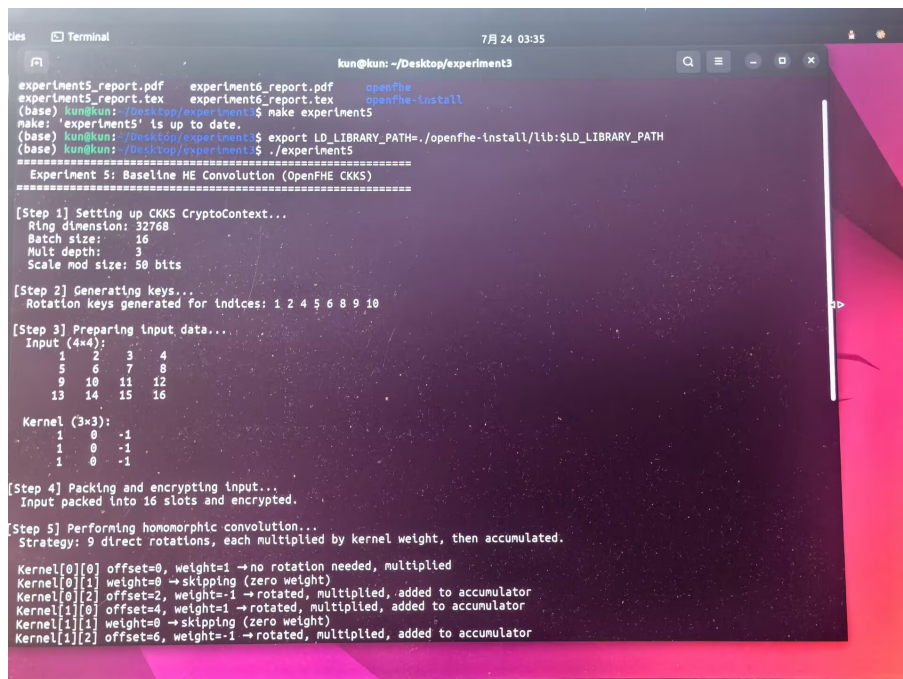
Max absolute error: 8.668621e-13

RESULT CORRECT! HE convolution matches plaintext convolution.

=====

4.2 实验截图

在本实验中获得的截图如下所示：



```
kun@kun: ~/Desktop/experiment3
experiment5_report.pdf  experiment6_report.pdf  openfhe
experiment5_report.tex  experiment6_report.tex  openfhe-install
(base) kun@kun: ~/Desktop/experiment3$ make experiment5
make: 'experiment5' is up to date.
(base) kun@kun: ~/Desktop/experiment3$ export LD_LIBRARY_PATH=./openfhe-install/lib:$LD_LIBRARY_PATH
(base) kun@kun: ~/Desktop/experiment3$ ./experiment5
=====
Experiment 5: Baseline HE Convolution (OpenFHE CKKS)
=====

[Step 1] Setting up CKKS CryptoContext...
Ring dimension: 32768
Batch size: 16
Mult depth: 3
Scale mod size: 50 bits

[Step 2] Generating keys...
Rotation keys generated for indices: 1 2 4 5 6 8 9 10

[Step 3] Preparing input data...
Input (4x4):
  1  2  3  4
  5  6  7  8
  9 10 11 12
 13 14 15 16

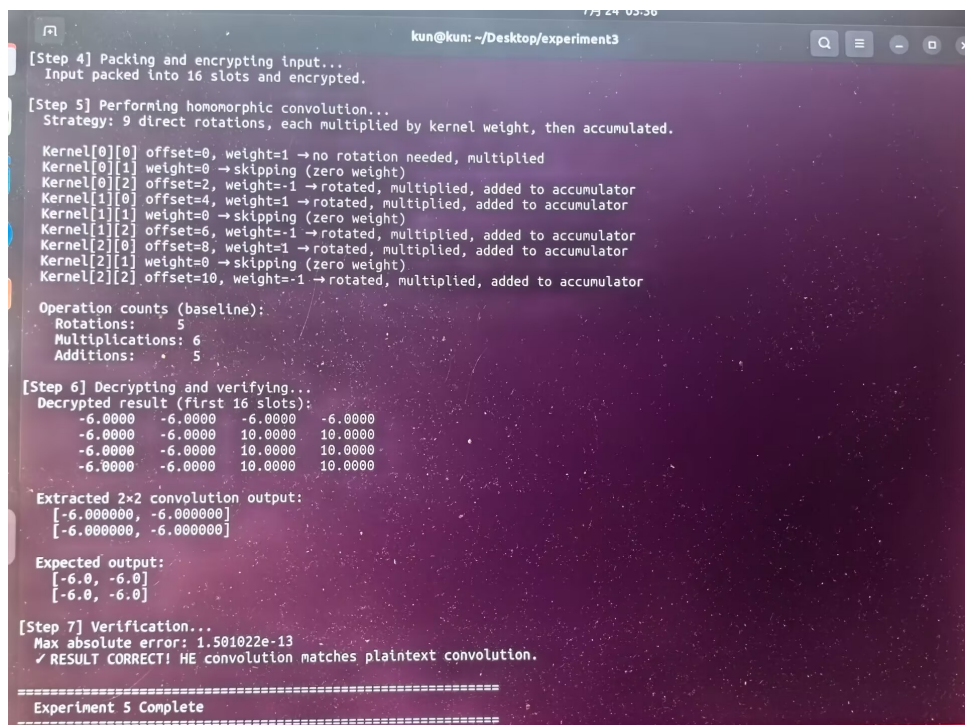
Kernel (3x3):
  1  0 -1
  1  0 -1
  1  0 -1

[Step 4] Packing and encrypting input...
Input packed into 16 slots and encrypted.

[Step 5] Performing homomorphic convolution...
Strategy: 9 direct rotations, each multiplied by kernel weight, then accumulated.

Kernel[0][0] offset=0, weight=1 → no rotation needed, multiplied
Kernel[0][1] weight=0 → skipping (zero weight)
Kernel[0][2] offset=2, weight=-1 → rotated, multiplied, added to accumulator
Kernel[1][0] offset=4, weight=1 → rotated, multiplied, added to accumulator
Kernel[1][1] weight=0 → skipping (zero weight)
Kernel[1][2] offset=6, weight=-1 → rotated, multiplied, added to accumulator
```

图 1: 实验截图 1



```
kun@kun: ~/Desktop/experiment3

[Step 4] Packing and encrypting input...
Input packed into 16 slots and encrypted.

[Step 5] Performing homomorphic convolution...
Strategy: 9 direct rotations, each multiplied by kernel weight, then accumulated.

Kernel[0][0] offset=0, weight=1 → no rotation needed, multiplied
Kernel[0][1] weight=0 → skipping (zero weight)
Kernel[0][2] offset=2, weight=-1 → rotated, multiplied, added to accumulator
Kernel[1][0] offset=4, weight=1 → rotated, multiplied, added to accumulator
Kernel[1][1] weight=0 → skipping (zero weight)
Kernel[1][2] offset=6, weight=-1 → rotated, multiplied, added to accumulator
Kernel[2][0] offset=8, weight=1 → rotated, multiplied, added to accumulator
Kernel[2][1] weight=0 → skipping (zero weight)
Kernel[2][2] offset=10, weight=-1 → rotated, multiplied, added to accumulator

Operation counts (baseline):
Rotations: 5
Multiplications: 6
Additions: 5

[Step 6] Decrypting and verifying...
Decrypted result (first 16 slots):
-6.00000 -6.00000 -6.00000 -6.00000
-6.00000 -6.00000 10.00000 10.00000
-6.00000 -6.00000 10.00000 10.00000
-6.00000 -6.00000 10.00000 10.00000

Extracted 2x2 convolution output:
[-6.000000, -6.000000]
[-6.000000, -6.000000]

Expected output:
[-6.0, -6.0]
[-6.0, -6.0]

[Step 7] Verification...
Max absolute error: 1.501022e-13
✓ RESULT CORRECT! HE convolution matches plaintext convolution.

=====
Experiment 5 Complete
=====
```

图 2: 实验截图 2

4.3 结果汇总

表 2: 实验结果汇总

指标	数值
输入尺寸	$4 \times 4 = 16$ slots
输出尺寸	$2 \times 2 = 4$ values
旋转密钥数量	8 个, 索引为 1, 2, 4, 5, 6, 8, 9, 10
实际旋转次数	5, 其中 3 个零权重被跳过
乘法次数	6
加法次数	5
最大绝对误差	8.669×10^{-13}
精度判定	✓ 正确, 误差远小于 10^{-10} 容忍阈值

5 误差分析

CKKS 是近似同态加密方案, 浮点数在以下环节产生舍入误差:

1. **编码**: 实数被缩放为整数并模 q , 此过程引入量化误差
2. **同态乘法**: 乘后的缩放因子为 Δ^2 , 需重缩放到 Δ , 损失低位精度
3. **解密/解码**: 将整数还原为浮点数时的舍入误差

本实验最大误差仅 8.669×10^{-13} , 相对误差约 2.7×10^{-12} , 证明了 CKKS 方案在此类计算中具有充足的精度余量。这得益于:

- 使用了 50 bits 的缩放模数
- 计算深度仅 1 层乘法, 精度损失极小
- 旋转和加法操作不消耗精度

6 解密结果的语义解释

解密后的 16 个槽位值为:

-6.0000	-6.0000	-6.0000	-6.0000
-6.0000	-6.0000	10.0000	10.0000
-6.0000	-6.0000	10.0000	10.0000
-6.0000	-6.0000	10.0000	10.0000

前 6 个有效值 -6 对应位置 $\{0, 1, 2, 4, 5, 6\}$, 其中 $\{0, 1, 4, 5\}$ 是有效的 2×2 输出区域。其余位置如 10 是算法在无效区域产生的副产品, 根据卷积定义 $O[i, j]$ 仅当 $i < 2$ 且 $j < 2$ 时有效。